

图像分类

1. 实验目的

- 1.1 了解 CIFAR-10 数据集;
- 1.2 掌握 GoogLeNet 网络的结构
- 1.3 完成 CIFAR-10 图像数据集的分类任务

2. 实验环境

SUB KIT NX 嵌入式开发板。

3. 准备工作

3.1 导入必要的模块

```
import torch                #导入构建网络模块
import torch.nn.functional as F    #导入激活函数模块
import torch.optim as optim    #导入优化器模块
from torchvision import transforms    #导入转换操作模块
from torchvision import datasets    #导入数据集模块
from torch.utils.data import DataLoader    #导入打包模块
```

3.2 构建小批量训练集和测试集

CIFAR-10 数据集由 60000 个分辨率为 32x32 的彩色图像组成，共分为 10 个类别，分别是 'airplane'、'automobile'、'bird'、'cat'、'deer'、'dog'、'frog'、'horse'、'ship'、'truck'，每个类别有 6000 个图像。

CIFAR-10 数据集中有 50000 个训练图像和 10000 个测试图像。数据集分为五个训练批次和一个测试批次，每个批次有 10000 个图像。测试批次在 10 个类别中分别随机选择 1000 个图像。训练批次以随机顺序包含剩余图像(一个训练批次中，各类图像的数量不一定相等)。CIFAR-10 数据集的内容如图 1 所示。

名称	修改日期	类型	大小
 batches.meta	2009/3/31 12:45	META 文件	1 KB
 data_batch_1	2009/3/31 12:32	文件	30,309 KB
 data_batch_2	2009/3/31 12:32	文件	30,308 KB
 data_batch_3	2009/3/31 12:32	文件	30,309 KB
 data_batch_4	2009/3/31 12:32	文件	30,309 KB
 data_batch_5	2009/3/31 12:32	文件	30,309 KB
 readme.html	2009/6/5 4:47	360 Chrome HT...	1 KB
 test_batch	2009/3/31 12:32	文件	30,309 KB

图 1 CIFAR-10 数据集内容

本次实验所用的数据集是 CIFAR-10，由于 torchvision 已经对许多常见数据集进行打包管理，故实验数据集可离线下下载后手动引入，也可在训练过程中联网下载。CIFAR-10 数据集大约 160M，在线下载需花费一定的时间，如果已经下载有 CIFAR-10，可通过 root 参数指定引用。

3.3 定义数据的预处理方式

```
transform = transforms.Compose([
    transforms.ToTensor(),          #输入除以 255，归一化，将输入归一化到(0,1)
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)), #使用“(x-mean)/std”，
    #将每个元素分布到(-1,1)，由于彩色图像为三通道，故 mean 和 std 均为 1*3 形式)
```

3.4 处理训练集数据

3.4.1 提取训练集数据

```
trainset = datasets.CIFAR10(
    root='G:\数据集\CIFAR10\cifar-10-python', #CIFAR-10 的保存路径
    train=True,                                #是否取训练集，是为 True
    download=False,                            #如果数据集未下载过，这里要改为 True
    transform=transform)                      #数据转换
```

3.4.2 打包训练集数据

```
trainloader = DataLoader(
    trainset,                                #被打包的数据
    batch_size=64,                           #一次训练所选取的样本数
    shuffle=True)                             #是否要打乱顺序，是为 True
```

3.5 处理测试集数据

3.5.1 提取测试集数据

```
testset = datasets.CIFAR10(  
    root='G:\数据集\CIFAR10\cifar-10-python',    #CIFAR-10 的保存路径  
    train=False,    #是否取训练集，否为 False  
    download=False,    #如果之前没下载数据集，这里要改为 True  
    transform=transform)    #数据转换
```

3.5.2 打包测试集数据

```
testloader = DataLoader(  
    testset,    #被打包的数据  
    batch_size=64,    #一次测试所选取的样本数  
    shuffle=False)    #是否要打乱顺序，是为 True
```

4. 构建网络模型

4.1 网络结构

卷积神经网络的结构多种多样，可以在网络的深度上进行延伸，也可在网络的宽度上进行拓展。GoogLeNet 采用了多个 Inception 模块来提升网络的深度和宽度，从而达到提高分类准确率的目的，本实验所用的网络是 GoogLeNet 的简化版。

4.1.1 Inception 模块结构

网络中的 Inception 模块由 4 个分支组成，其具体结构如图 2 所示，输入数据分别由 4 个分支进行处理（处理前后图像尺寸一样），然后将 4 个分支的输出堆叠在一起作为下一层的输入。

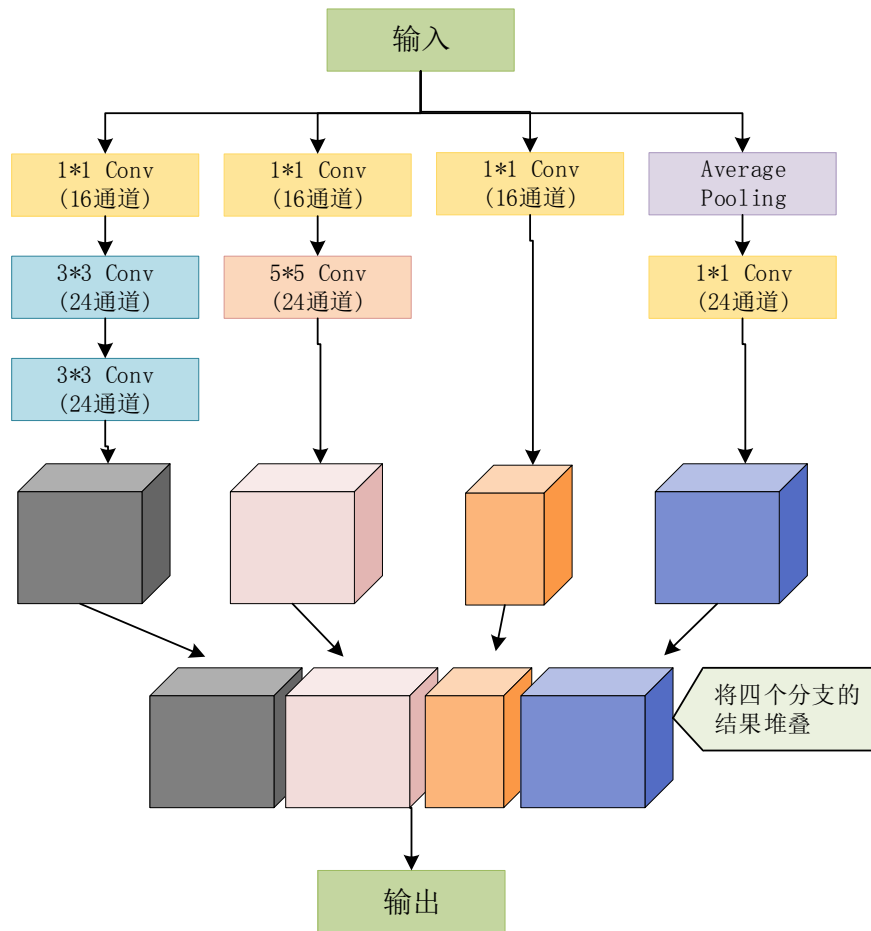


图2 Inception 模块结构

用类的方式定义 Inception 模块：

torch.nn.Conv2d (输入通道，输出通道，卷积核尺寸，图像边缘填充 0 的圈数)

#图像尺寸计算公式：[(图像尺寸 - 模板尺寸 + 2*填 0 圈数) / 步长] + 1

```
class InceptionA(torch.nn.Module):
```

```
    def __init__(self,in_channels):
```

```
        super(InceptionA,self).__init__()
```

```
        self.branch3x3_1=torch.nn.Conv2d(in_channels,16,kernel_size=1)
```

```
        self.branch3x3_2=torch.nn.Conv2d(16,24,kernel_size=3,padding=1)
```

```
        self.branch3x3_3=torch.nn.Conv2d(24,24,kernel_size=3,padding=1)
```

```
        self.branch5x5_1=torch.nn.Conv2d(in_channels,16,kernel_size=1)
```

```
        self.branch5x5_2=torch.nn.Conv2d(16,24,kernel_size=5,padding=2)
```

```

self.branch1x1=torch.nn.Conv2d(in_channels,16,kernel_size=1)

self.branch_pool=torch.nn.Conv2d(in_channels,24,kernel_size=1)

def forward(self,x):
    #分支 1 处理过程
    branch3x3=self.branch3x3_1(x)
    branch3x3=self.branch3x3_2(branch3x3)
    branch3x3=self.branch3x3_3(branch3x3)
    #分支 2 处理过程
    branch5x5=self.branch5x5_1(x)
    branch5x5=self.branch5x5_2(branch5x5)
    #分支 3 处理过程
    branch1x1=self.branch1x1(x)
    #分支 4 处理过程
    branch_pool=F.avg_pool2d(x,kernel_size=3,stroke=1,padding=1)
    branch_pool=self.branch_pool(branch_pool)
    #将 4 个分支的输出拼接起来
    outputs=[branch1x1,branch5x5,branch3x3,branch_pool]
    return torch.cat(outputs,dim=1) #横着拼接，共有 24+24+16+24=88 个通道

```

4.1.2 网络整体结构

整个网络的具体结构如图 3 所示，网络按照“卷积层 1—池化层—Inception—卷积层 2—池化层—Inception—全连接层”的顺序进行连接。

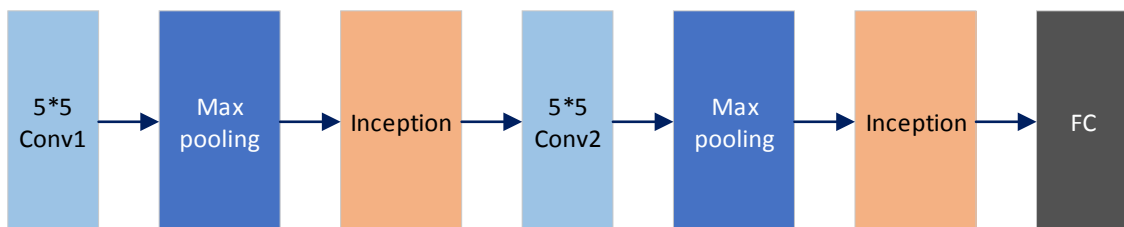


图 3 网络整体结构

用类的方式定义网络：

```
class Net(torch.nn.Module):
```

```

def __init__(self):
    super(Net,self).__init__()
    self.conv1=torch.nn.Conv2d(3,10,kernel_size=5)
    self.conv2=torch.nn.Conv2d(88,20,kernel_size=5)
    ###Inception 模块的输出均为 88 通道
    self.incep1=InceptionA(in_channels=10)
    self.incep2=InceptionA(in_channels=20)

    self.mp=torch.nn.MaxPool2d(2)

    self.fc=torch.nn.Linear(2200,10)    #5*5*88=2200, 图像高*宽*通道数

def forward(self,x):
    x=F.relu(self.mp(self.conv1(x)))    #图像尺寸 14*14
    x=self.incep1(x)                    #图像尺寸 14*14
    .....
    return x

```

4.1.3. 实例化网络并定义优化器

完成对网络的定义后，需要把网络实例化，同时定义训练过程中所用的优化器。本次实验采用的损失函数为交叉熵损失函数，优化方式为随机梯度下降法(SGD)。

```

model=Net()
model=model.cuda()                #将模型迁移至 GPU
criterion=torch.nn.CrossEntropyLoss()    #损失函数
optimizer=optim.SGD(model.parameters(),lr=0.01,momentum=0.5)#SGD，学习率为
0.01，动量为 0.5

```

5. 定义训练网络

定义一个用于训练的函数，通过循环来实现训练操作，

```

def train(epoch):
    running_loss=0.0
    for batch_idx,data in enumerate(trainloader,0):    #枚举

```

```
..... #训练程序
```

```
.....#打印训练结果，每 300 个 batch 打印一次
```

6. 定义测试网络

定义一个用于测试网络性能的函数，训练完成后，用测试集来测试网络在图像分类方面的能力。注意：测试的过程只包含前向传播，不包含后向传播操作。

```
def test():
```

```
    correct=0
```

```
    total=0
```

```
    # 由于测试的时候不要求导，可以暂时关闭 autograd，提高速度，节约内存
```

```
    with torch.no_grad():
```

```
        for data in testloader:
```

```
            images,labels=data          #读取测试数据
```

```
            images = images.cuda()      #将测试集数据迁移至 GPU
```

```
            labels = labels.cuda()      #将测试集数据迁移至 GPU
```

```
            outputs=model(images)       #前向传播
```

```
            predicted=torch.max(outputs.data,dim=1) #返回每一行的最大值，以及最大值的下标
```

```
            total+=labels.size(0)       #样本总数
```

```
            correct+=(predicted == labels).sum().item() #正确匹配的数量
```

```
    print('Accuracy on test set: %d %%%'%(100*correct/total)) #打印正确率
```

7. 开始训练和测试网络

定义好训练过程和测试过程后，就可以开始训练和测试网络了，这里仅将全部样本训练 3 次(也可增加全部样本的训练轮数，但是训练所消耗的时间也会延长)。

```
if __name__=='__main__':
```

```
    for epoch in range(3):
```

```
        train(epoch)                    #epoch: 可自定义训练轮数
```

```
        test()
```

8. 实验任务

同学编写 GoogLeNet 网络，实现 CIFAR-10 数据集的分类任务。